

Lab Report: Lab 1 - Requirements Elicitation & Modeling (OminiDash)

Contents

1. Cover Page	1
2. Abstract/Summary	1
3. Lab Specific Section: I. Requirements Elicitation & Modeling.....	2
3.1 Software Requirements Specifications (SRS).....	2
3.2 Modeling Artifact: UML Use Case Diagram.....	4
4. Architectural Design (<i>note: Problem analysis of next lab</i>)	4
5. Conclusion & Reflection	5

1. Cover Page

- **Title:** Elicitation and Modeling Requirements for the OmniDash Personal Productivity Platform
- **Course Info:** Software Architecture
- **Student Details:** Nguyễn Văn Trường, 23010371
- **Date:** April 10, 2026

2. Abstract/Summary

The **OmniDash** project aims to develop a comprehensive **personal productivity platform**. This initial lab focused on requirements elicitation and modeling for a multi-context digital workspace that balances work, life, and entertainment.

We documented essential **Functional, Non-Functional, and Architecturally Significant Requirements (ASRs)** in a structured format, highlighting the system's unique ability to manage "Workspace Combos" and context-driven workflows. The core system behavior was formally modeled using a **UML Use Case Diagram**, clearly defining system boundaries between different user modes, external actors (such as AI services and browser extensions), and key behavioral relationships.

This foundation establishes the scope and the critical architectural drivers—including **real-time synchronization and multi-platform integration**—that will be addressed in subsequent labs, particularly the design of the **Layered Architecture** in Lab 2.

3. Lab Specific Section: I. Requirements Elicitation & Modeling

This section documents the deliverables for Lab 1, as specified in the report requirements.

3.1 Software Requirements Specifications (SRS)

The following tables document the elicited requirements, which collectively form the basis for the OmniDash design.

3.1.1 Functional Requirements (FRs)

These requirements define the specific behaviors and functions the OmniDash system must provide.

ID	Description	Priority
FR-01	The system must allow a User to create, manage, and launch " Workspace Combos " (pre-defined groups of URLs and resources) via a browser extension integration.	Critical
FR-02	The system must provide a Task Management module allowing users to perform CRUD operations on tasks with support for Kanban, List, and Calendar views.	High
FR-03	The system must support Context Switching , allowing users to toggle between different modes (e.g., Work, Personal, Chill) with unique dashboard layouts and widget configurations.	High
FR-04	The system must integrate AI Assistance to automatically decompose complex tasks into actionable sub-tasks and provide semantic search across notes.	High
FR-05	The system must provide a Rich-text/Markdown Note-taking utility that allows users to capture, categorize, and link personal ideas or project documentation.	Medium

3.1.2 Non-Functional Requirements (NFRs)

These requirements specify criteria that can be used to judge the operation of the system, not specific functions.

ID	Attribute	Description	Impact
NFR-01	Performance (Latency)	The system must load the personalized dashboard and "Workspace Combos" within 1.5 seconds under normal load to ensure a seamless flow.	High
NFR-02	Security (Integrity)	All user data, especially personal notes and workspace URLs, must be transmitted via HTTPS and stored using strong encryption (e.g., AES-256).	Critical

NFR-03	Reliability (Availability)	The system must maintain a minimum of 99.5% uptime to ensure users can access their productivity tools whenever needed.	Critical
NFR-04	Usability	The interface must follow Minimalist Design principles to reduce cognitive load, with context-switching (e.g., Work to Chill mode) occurring in under 500ms .	High
NFR-05	Scalability	The backend architecture must be able to handle a concurrent increase in users and AI-processing requests without significant degradation in response time.	

3.1.3 Architecturally Significant Requirements (ASRs)

These are the non-functional requirements that have the most profound influence on the system's architecture, driving the selection of patterns.

ASR ID	Quality Attribute	Requirement Statement	Architectural Rationale
ASR-1	Intergability	The architecture must support seamless, bi-directional communication between the Web Application and the Browser Extension for launching "Workspace Combos."	This necessitates a well-defined API Layer and a secure message-passing protocol to ensure the extension can execute browser actions based on backend data.
ASR-2	Extensibility	Adding new utility widgets (e.g., AI Assistant, Habit Tracker, or RSS Feeds) must not require modifications to the core Task Management or Auth components.	Directly enforces the Separation of Concerns (SoC) principle, driving the design toward a Layered or Component-based architecture where utilities are decoupled from core logic.
ASR-3	Performance	The system must handle Asynchronous AI Processing (e.g., task decomposition) without blocking the main user interface or UI responsiveness.	Requires an architecture that supports Asynchronous Task Queues or non-blocking API calls to ensure a smooth user experience while waiting for AI responses.

3.2 Modeling Artifact: UML Use Case Diagram

The UML Use Case Diagram below models the functional scope of the **OmniDash** system, showing the system boundary, the external **Actors**, and the core Use Cases.

- **Actors:** The diagram identifies two human actors and two supporting system actors: User, Admin,

AI Service, Browser Extension.

- **System Boundary:** The box delineates the scope of the **OmniDash Platform**, separating internal logic from external entities.
 - **Core Use Cases:**
 - For **User: Manage Workspace, Switch Context, Manage Tasks,** and **Generate AI Insights.**
 - For **Admin: Manage User Accounts, Monitor System Analytics,** and **Manage AI API Configurations.**
 - **Critical Path Flow:** The diagram details the **Manage Workspace** process for the User and the **System Monitoring** process for the Admin, ensuring the platform remains stable and responsive.
 - **Relationships Flow:**
 - **Include Relationship:** The "Select Payment Method" Use Case is a mandatory part of the **Checkout** process, modeled using the "include" relationship.
 - **Extend Relationship:** The "Apply Coupon Code" Use Case is an optional behavior that extends the **Checkout** process, modeled using the "extend" relationship.
-

4. Conclusion & Reflection

The requirements elicitation phase for **OmniDash** successfully defined the project scope through detailed Functional, Non-Functional, and Architecturally Significant Requirements (ASRs). The UML Use Case Diagram provided a clear visual model of user interactions and identified the necessary boundaries between the platform and external actors, such as the **AI Service** and **Browser Extension**. These identified ASRs—specifically regarding integrability, extensibility, and asynchronous performance—directly necessitate the adoption of a structured **Layered Architecture** for the subsequent design phase in Lab 2. This foundation ensures that the platform can sustainably support complex behaviors while maintaining high performance and responsiveness as the project evolves.